

Local-first v3 combined cutover

What changed in the refactor, why the new path is safer under failure, and how to operate or roll it back without losing accepted user intent.

Production flags live

31 Jul 2026

Primary commit 2a56e598

Rollback fix fa085f87

EXECUTIVE SUMMARY

We replaced the supported web slices' split optimistic-and-query behavior with one durable loop: v3 materialized reads plus a receipt-backed command journal. The cutover is intentionally aggressive, but reversible. Legacy tables and handlers remain, accepted commands are retained, and one master flag returns supported web reads and newly promoted writes to their prior posture.

THE NEW LOOP



BEFORE AND AFTER

Before: two loosely coupled paths

- Legacy query results were merged with in-memory optimistic state.
- Some writes were fire-and-forget and could outlive their UI signal.
- Retries, rejection rollback, and echo matching varied by action.
- Offline intent durability was implied, not explicitly gated.

After: one replayable correctness loop

- v3 reads are durable, principal-scoped materialized views.
- Supported writes carry stable command identities and receipts.
- Unknown journal schemas park instead of being guessed or erased.
- Storage denial is a visible degraded mode, not false durability.

RELEASE SCOPE

Buckets + comments
v3 log-ts read contracts and durable write coverage.

Message send
Must-deliver send/v2 with exact client identity propagation.

Convex substrate
Additive handlers, receipts, view echoes, legacy compatibility.

Web rollback
One master fail-closed flag; durable rows remain drainable.

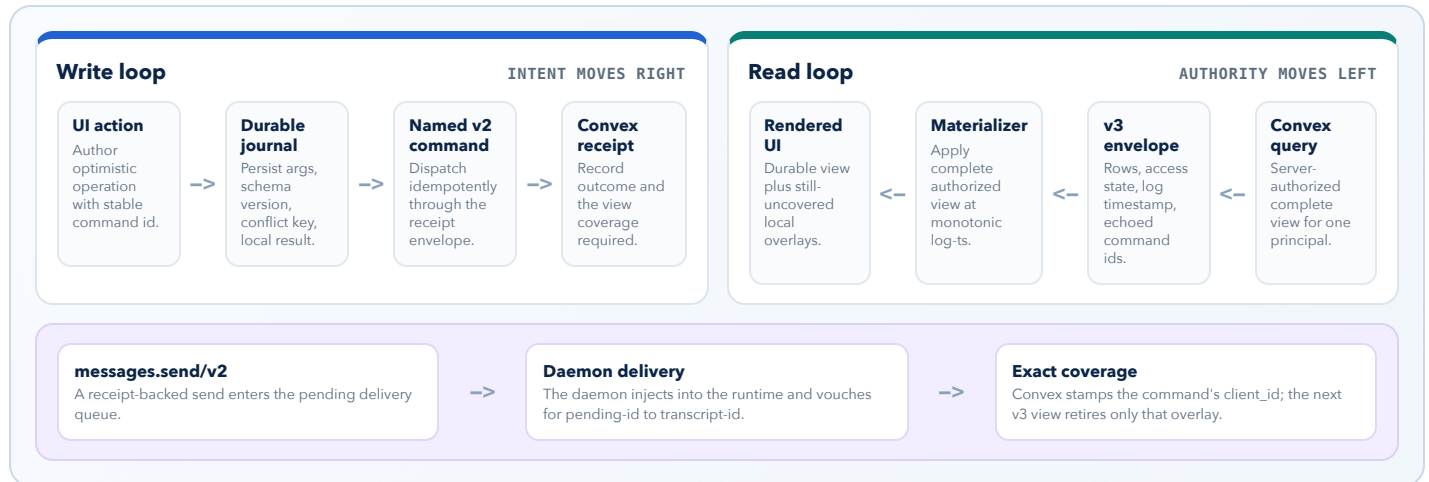
Fenced execution
Code shipped for conversation-scoped, mixed-version-safe activation.

Native/mobile
Deliberately stays on v1; no mobile binary required.

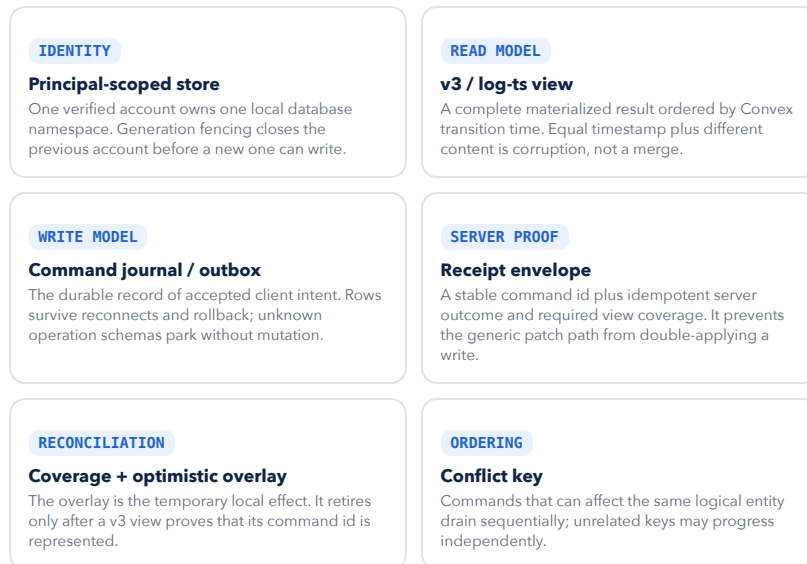
How the pieces fit together

The architecture separates four responsibilities: authorize the local principal, durably record intent, commit it idempotently on the server, and prove that the durable read model has absorbed it. The optimistic UI is temporary; the receipt and v3 view are authoritative.

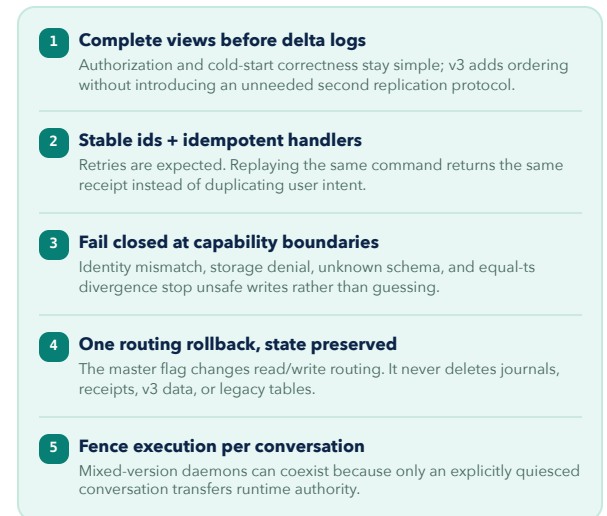
TWO LOOPS, ONE CONVERGENT STATE



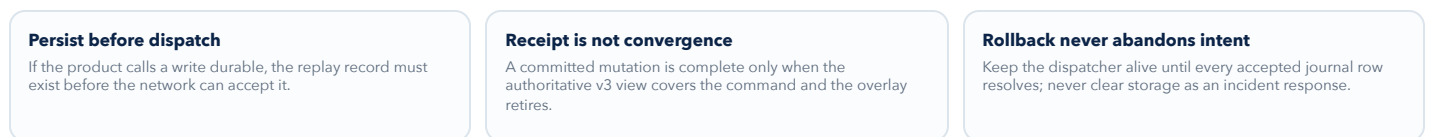
KEY TERMS



WHY THESE CHOICES



THREE INVARIANTS TO REMEMBER



How reads and writes converge

The refactor makes the client prove both identity and durability before it claims an offline-capable write, then uses a server receipt and v3 view coverage to retire the optimistic overlay.

WRITE PATH

1

Author concrete optimistic state

The store captures a stable command id and a replayable operation, not an opaque callback.

2

Persist before network dispatch

The outbox row carries `operationSchemaVersion=1`, action args, conflict key, local result, and receipt envelope.

3

Execute an idempotent named command

Convex dispatch routes the final action to the v2 handler and skips generic legacy patches for that envelope.

4

Observe receipt coverage

The v3 envelope returns a log timestamp plus recently acknowledged command ids for that view key.

5

Reconcile or reject exactly

Coverage retires the overlay. A permanent rejection rolls back only values still owned by that optimistic command.

CLOSED RELEASE GATES

R-01

Persistent storage posture

After verified boot, the browser requests persistent storage. If denied or unsupported, reads and online writes remain available, but offline final writes are refused before optimistic paint or enqueue.

CMD-02

Journal schema fold gate

Every command read, hydration, fold, and write boundary checks the operation schema. Explicit legacy v0 is understood; unknown future shapes remain durable and parked for migration.

R-11

No temp-id dependency chain

Create-bucket-with-assignments is one atomic receipt command. A temporary bucket id never enters a second queued assignment that cannot be rewritten safely.

DWB-03

Exact send acknowledgment

The daemon vouches for pending-id to transcript-id delivery. Convex validates ownership and role, then stamps the exact client id before terminal delivery.

READ PATH

Complete v3 view, stamped at one Convex transition

The buckets and per-conversation comments contracts materialize the complete authorized result in IndexedDB. Ordering uses Convex log timestamps. An equal timestamp with different content or access is treated as corruption, not silently merged.

Why echoed ids exist: today, Convex mutation resolution does not expose the exact commit timestamp to the browser. Echoing acknowledged command ids in the same view snapshot is the sound bridge until that platform capability exists.

Principal safety: local databases, launcher state, generation fencing, logout purge, and legacy quarantine are scoped to the verified account. A token transition closes dispatch synchronously before the old principal can accept another click.

Boot-window UX remains operationally important: dispatch can park while auth or bindings initialize, and the inspector exposes degraded state. Teammates should treat missing user feedback here as a product bug even when the durable row is safe.

Reversible by construction

The production risk was accepted, but irreversible data loss was not. The server additions are backward-compatible, legacy paths remain present, and rollback changes authoring and reads without deleting accepted intent.

SERVER AND DAEMON CHANGES

CONVEX DISPATCH

Receipt envelope owns the final write

Final actions route to named v2 handlers. Generic patches are skipped for receipt envelopes, preventing the same logical write from traveling through two correctness systems.

BUCKETS AND COMMENTS

Idempotent commands with view coverage

Create, update, assignment, and comment actions record receipts that name the v3 contract and view key required to reconcile the optimistic state.

MESSAGES.SEND/V2

Identity resolved before duplicate suppression

Two same-content commands remain distinct when client ids differ. Truncated or duplicate-suppressed echoes no longer leave coverage unprovable.

DAEMON ACKNOWLEDGMENT

Pending id maps to exact transcript id

The ack path cannot overwrite another command's client id, and it validates conversation, author role, and principal before stamping delivery coverage.

ONE ROLLBACK RAIL

Final mode on

V2_ENABLED=1 + FINAL_MODE=1

Buckets/comments read
cutover-lts;
newly promoted update, assign, and send actions use receipt envelopes.



Master off

VITE_LOCAL_FIRST_V2_ENABLED=0

Supported reads return to v1. Final-mode promotions return to their exact pre-release posture while accepted outbox rows continue draining.

Fix-forward correction fa085f87: rollback must not demote actions that were receipt-backed before final mode. Comment actions and createBucket keep their established rejection reconciliation in both postures; only updateBucket, assignSessionToBucket, and sendMessage are master-flag promotions.

Rollback preserves

- Accepted outbox and journal rows.
- Receipt tables, v3 materialized data, and legacy tables.
- The dispatcher needed to finish already accepted commands.
- Previously shipped receipt semantics and rejection rollback.

Rollback never means

- Deleting IndexedDB or server rollback state.
- Guessing an unknown operation schema.
- Stranding pending intent by disabling its only dispatcher.
- Removing legacy tables during the observation window.

PLATFORM BOUNDARIES

Fenced execution rail

The coordinator and four execution tables are shipped, but activation is local-flagged and conversation-scoped. Legacy daemons remain responsible for legacy conversations. This avoids mixed-version split brain.

CLI and fleet rollout

Web final mode does not force a CLI minimum. Source support is committed; a signed CLI publication and live restart/orphan fenced proof remain separate operational gates.

Mobile and small views

Native resolves useSyncBuckets.native.ts, stays on the v1 subscription, and never sees Vite flags. No iOS build is needed. Unmaterialized small views also remain fail-closed.

What shipped, what was proven, what remains

This is the handoff page: the release sequence, concrete verification, production observations, and the shortest safe runbook for the next engineer.

RELEASE SEQUENCE

29 JUL

Additive substrate + shadow

Principal store, typed materializer, v2 envelopes, and digest comparison soaked with zero recorded mismatches.

30 JUL

Combined direction confirmed

Incremental flips were replaced by one reversible final-mode release. No prior irreversible work was racing.

30 JUL

2a56e598 deployed

Convex first, then web final flags. 26 files; 794 insertions and 46 deletions.

30-31 JUL

Rollback corrected + inherited

fa085f87 fixed receipt demotion. Current successful web deploys descend from the cutover line; final flags remain enabled.

VERIFICATION RECORDED FOR THE CUTOVER

969

Convex tests passed

62

Selected CLI sync / ack / execution tests

59

Focused final-mode web tests

3

Package typechecks clean: web, Convex, CLI

Confirmed

- LIVE** Convex deployed successfully before the web cutover.
- LIVE** Production has `V2_ENABLED=1` and `FINAL_MODE=1`; slice shadow variables are overridden to `cutover-lts`.
- BUILT** Final-mode production web build and PWA generation completed successfully.
- SAFE** No legacy table, rollback state, public minimum, forced CLI release, or mobile binary was deleted or changed.

Known proof gaps and observed risk

- OPEN** A fully authenticated final-mode browser write smoke was not completed: the signed-in custom origin retained an older PWA shell while an alternate origin could not reuse OAuth.
- OPEN** Rapid Railway deploy churn removed an old lazy chunk, producing a real dynamic-import failure. Asset retention and immediate service-worker update need fix-forward work.
- OPEN** The fenced rail still needs one clean production conversation proof, including daemon restart and orphan recovery, before any default-on fleet change.

OPERATOR RUNBOOK

If production degrades

1. Check persistent-storage posture, parked/rejected outbox rows, send pending age, and equal-ts corruption signals.
2. Prefer fix-forward while accepted commands continue draining.
3. If rollback is required, set `VITE_LOCAL_FIRST_V2_ENABLED=0`. Do not clear client storage or delete server tables.
4. Keep the dispatcher active until every accepted v2/v3 journal row is resolved.

Next high-value work

1. Run an authenticated fresh-PWA final-mode write smoke for bucket update, assignment, comment, and send.
2. Complete the conversation-scoped fenced E2E with restart and orphan recovery.
3. Publish a signed CLI only if fleet propagation is required; do not substitute an ad-hoc unsigned macOS binary.
4. After the rollback observation window, delete legacy paths only through an explicit gated task.

Primary sources: commits 2a56e598 and fa085f87; docs/architecture/local-first-v3-rollout-pipeline.md; local-first-sync-test-matrix.md; local-first-sync-and-runtime-binding-design.md. Production flags and latest successful Railway status rechecked 31 Jul 2026.